MY FIRST TETRIS IN PYTHON

Start the game: `python3 tetris.py`

Keys: ← → to move, ↑ to rotate, ↓ to drop faster.

PROJECT LAYOUT_

FILE / FOLDER	WHAT IT IS
`tetris.py`	The main file: the game loop and the keys. Run this one.
`config.py`	**Exercise 1**: the game settings
`pieces/`	**Exercise 2**: one file per piece
`utils/helpers.py`	**Exercise 3**: scoring (`clear_lines`), plus the behind-the-scenes functions
`moves.py`	**Bonus**: the moves of the piece (`rotate_back` is empty)
`utils/state.py` , `utils/display.py`	Game variables and drawing: no need to touch them

EXERCISE 1: CUSTOMIZE YOUR GAME (10 MIN)_

In `config.py`, change the values and restart the game:

- `SPEED`: make a very slow game (1000), then a very fast one (100)
- `BACKGROUND_COLOR`: try “navy”, “white”, “pink”
- `WIDTH` / `HEIGHT` / `CELL_SIZE`: make a giant or a tiny game

Concept: a **variable** is a box with a name that holds a value.

EXERCISE 2: CREATE YOUR OWN PIECE (15 MIN)

A piece is a file in the `pieces/` folder with a list of squares `(x, y)` and a color. Draw it on paper first! Ideas: the S piece, a single dot, a big cross, a line of 5, your initial...

1. Copy a file (for example `t_shape.py`) and give it a new name.
2. Change `SQUARES` and `COLOR` in your new file.
3. In `pieces/__init__.py`, import it and add it to the `PIECES` list.

Concepts: a **list**, **coordinates**, the comma that separates items, **importing** a file.

EXERCISE 3: THE SCORE (15 MIN)

In `utils/helpers.py`, in the function `clear_lines`, the variable `nb` is the number of lines just removed. Give points to the player with `state.score = state.score + ...`

- Bonus 1: more points when several lines are removed at once (`if nb == 4:`)
- Bonus 2: the game speeds up with every line (`state.delay = state.delay - 20`)

Concepts: **changing a variable**, a **calculation**, a **condition** (`if`).

BONUS: ROTATE THE OTHER WAY (FOR THE FASTEST)

In `moves.py`, the function `rotate_back` is empty. The **z** key already calls it, so you only have to write the function.

Hint: `rotate` turns each square `(x, y)` into `(-y, x)`. What should the new square be to turn the other way? Draw a piece on paper, turn it, and compare the coordinates. Don't forget to check for a `collision` before changing the piece!

Concepts: **functions**, **reading existing code**, **checking before acting**.